

Spectral Graph Lab

1. 專題定位

Project Type: Mathematical Machine Learning / Spectral Graph Theory / Graph Learning Foundations

核心技術建議：

- Python
- NumPy
- SciPy
- NetworkX
- Plotly
- Streamlit

核心問題：

How does the spectrum of a graph encode its geometry, connectivity, and cluster structure?

本專題希望建立一個互動式的 Spectral Graph Laboratory。

使用者不只是看到 spectral clustering 的最後結果，而是可以從建立 graph 開始，逐步觀察：

Graph Structure $\rightarrow$ Adjacency / Degree Matrix $\rightarrow$ Graph Laplacian $\rightarrow$ Spectrum $\rightarrow$ Fiedler Vector $\rightarrow$ Connected Components

專題的主要價值在於將原本抽象的 spectral graph theory 概念轉化為可以「直接操作」的數學直覺。

2. 專題核心學習目標

使用者在完成互動體驗之後，應該能理解以下問題：

1. Graph 如何被轉換成 matrix？
 2. Graph Laplacian 是什麼？
 3. 為什麼 Graph Laplacian 的 eigenvalues 與 connectivity 有關？
 4. 第二小 eigenvalue λ_2 為什麼稱為 algebraic connectivity？
 5. Fiedler vector 是什麼？
 6. 為什麼 Fiedler vector 可以揭露 graph partition？
 7. 為什麼 spectral clustering 可以處理一般 K-Means 不容易處理的幾何結構？
 8. Graph eigenvectors 為什麼可以被視為 graph 上的「frequency modes」？
 9. Graph Laplacian 與 continuous Laplacian / Laplace–Beltrami operator 有什麼概念上的對應？
 10. Spectral methods 如何成為 Graph Learning 與 spectral GNN 的數學基礎？
-

3. 整體網站架構

網站建議分成四個主要模組：

Module A — Graph Playground

讓使用者自行建立 graph，並即時計算 Laplacian、spectrum、algebraic connectivity 與 Fiedler vector。

主要負責：

Graph $\rightarrow$ Spectrum

Module B — Fiedler & Connectivity Lab

專門探索：

λ_2, v_2

與 connectivity、bottleneck、graph partition 的關係。

主要負責：

Spectrum $\rightarrow$ Connectivity

Module C — Spectral Clustering Lab

使用幾何資料建立 similarity graph，並比較：

K-Means vs. Spectral Clustering.

主要負責：

Geometry $\rightarrow$ Graph $\rightarrow$ Spectral Embedding $\rightarrow$ Clustering

Module D — Eigenfunction / Mesh Lab

使用 toy mesh 或公開 3D mesh 展示 Laplacian eigenvectors / eigenfunctions。

主要負責：

Discrete Laplacian $\leftrightarrow$ Continuous Laplacian

並建立通往 Graph Fourier Analysis 與 Spectral GNN 的概念橋樑。

4. Module A — Graph Playground

4.1 功能目標

讓使用者能夠直接建立一張 graph。

使用者可以：

- 新增 node
 - 刪除 node
 - 建立 edge
 - 刪除 edge
 - 拖動 node 位置
 - 選擇 edge 是否有 weight
 - 修改 edge weight
 - 清空 graph
 - 載入 predefined graph
-

5. Graph Playground 的重要設計原則

需要區分：

Graph Layout

以及

Graph Geometry

兩者不能混在一起。

5.1 Free Graph Mode

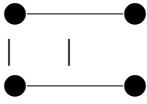
在 Free Graph Mode 中：

node 的畫面位置只負責 visualization。

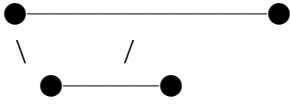
例如：

$$V = \{1, 2, 3, 4\}$$

edge structure 固定時，即使使用者拖動 node：



變成：



如果 adjacency relationship 沒有改變，則 graph 在數學上仍然相同。

因此：

$$A, \quad D, \quad L$$

不應該因為畫面上的 node position 改變。

這個設計應該明確反映：

Graph topology does not depend on visual layout.

6. Free Graph Mode 即時計算內容

每當 graph structure 改變時，自動重新計算：

Adjacency Matrix

$$A$$

Degree Matrix

$$D$$

Unnormalized Laplacian

$$L = D - A$$

Symmetric Normalized Laplacian

$$L_{\text{sym}} = I - D^{-1/2}AD^{-1/2}$$

Random-Walk Laplacian

$$L_{\text{rw}} = I - D^{-1}A$$

使用者可以切換 Laplacian Type :

- Unnormalized
- Symmetric Normalized
- Random Walk

7. Spectrum Visualization

計算：

$$Lv_i = \lambda_i v_i$$

並將 eigenvalues 由小到大排序：

$$0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n.$$

畫面顯示：

- 完整 eigenvalue list
- eigenvalue spectrum plot
- number of zero eigenvalues
- λ_2
- algebraic connectivity
- selected eigenvector

例如：

Eigenvalue Spectrum

```
 $\lambda_1 = 0.000$   
 $\lambda_2 = 0.137$   
 $\lambda_3 = 1.241$   
 $\lambda_4 = 2.000$   
...
```

8. Connected Components Experiment

Graph Laplacian 的一個重要概念是：

multiplicity of eigenvalue 0 = number of connected components.

因此 UI 可以即時顯示：

Connected Components: 2
Zero Eigenvalues: 2

讓使用者自己新增或刪除 bridge edge。

例如：

Cluster A — Cluster B

刪除中間 bridge 後：

Cluster A Cluster B

使用者應該能觀察到：

$$\lambda_2 > 0$$

變成：

$$\lambda_2 = 0.$$

9. Module B — Fiedler & Connectivity Lab

本模組是整個 Spectral Graph Lab 的核心。

10. Algebraic Connectivity

對 connected graph：

$$0 = \lambda_1 < \lambda_2.$$

其中：

$$\lambda_2$$

稱為 algebraic connectivity。

使用者修改 graph 時即時計算：

$$\lambda_2.$$

並顯示：

Algebraic Connectivity

$$\lambda_2 = 0.083$$

可以附上一個簡單 qualitative indicator，例如：

Connectivity:

Weak —●— Strong

但需註明：

這只是視覺輔助，不應將不同規模 graph 的 λ_2 直接當成完全可比較的 connectivity score。

11. Fiedler Vector

對應 λ_2 的 eigenvector：

$$v_2$$

稱為 Fiedler vector。

例如：

$$v_2 = [-0.43 \quad -0.39 \quad -0.31 \quad 0.28 \quad 0.37 \quad 0.41] .$$

每個 node i 的值：

$$v_2(i)$$

直接映射為 node color。

12. Fiedler Vector Visualization

建議同時提供三種 visualizations。

A. Graph Coloring

根據：

$$v_2(i)$$

將每個 node 連續上色。

負值與正值應該清楚區分。

B. Fiedler Coordinate Plot

每個 node 在一維座標軸上的位置為：

$$x_i = v_2(i).$$

這可以讓使用者理解：

Fiedler vector 可以被看成將每個 vertex 映射到一個一維 spectral coordinate。

C. Value Bar Plot

例如：

Node 1	-0.43
Node 2	-0.39
Node 3	-0.31

Node 4	0.28
Node 5	0.37
Node 6	0.41

13. Threshold Partition

提供 threshold：

$$t$$

預設：

$$t = 0.$$

將 graph 分成：

$$C_1 = \{i : v_2(i) < t\}$$

以及

$$C_2 = \{i : v_2(i) \geq t\}.$$

讓使用者可以拖動 threshold。

Graph partition 結果即時更新。

14. Smoothness 的數學解釋

本專題需明確解釋：

對任意 vector：

$$f = [f(1) : f(n)] \in \mathbb{R}^n,$$

也可以將其理解為：

$$f : V \rightarrow \mathbb{R}.$$

也就是對 graph 的每個 vertex assign 一個 scalar value。

因此：

$$\text{vector in } \mathbb{R}^n$$

與

$$\text{real-valued function on vertices}$$

是相同的數學物件，只是 interpretation 不同。

15. Graph Smoothness

對 graph function f ，定義：

$$f^T L f = \frac{1}{2} \sum_{i,j} w_{ij} (f(i) - f(j))^2.$$

這個 quantity 衡量：

相連 vertices 之間的 function values 是否接近。

如果：

$$f(i) \approx f(j)$$

對大部分 connected vertices 成立，則：

$$f^T L f$$

較小。

稱為：

f is smooth on the graph.

16. Smoothness Interactive Experiment

加入一個可以讓使用者自行修改 node values 的 toy experiment 。

例如 graph：

1 — 2 — 3 — 4 — 5

使用者可以設定：

$$f = (1, 1.1, 1.2, 1.3, 1.4).$$

顯示：

Graph Smoothness Energy:
Low

再改成：

$$f = (1, -1, 1, -1, 1).$$

顯示：

Graph Smoothness Energy:
High

並同時計算：

$$f^T L f.$$

目的不是新增一個獨立演算法，而是讓使用者直覺理解：

$$f^T L f$$

代表什麼。

17. Fiedler Vector 的 Variational Interpretation

對 normalized eigenvector：

$$\|v_k\| = 1,$$

有：

$$v_k^T L v_k = \lambda_k.$$

第一個 eigenvector :

$$v_1 \propto \mathbf{1}$$

是 constant vector 。

因為：

$$L\mathbf{1} = 0.$$

它在 graph 上完全沒有 variation 。

因此：

$$\lambda_1 = 0.$$

第二個 eigenvector 滿足：

$$\lambda_2 = \min_{f \perp \mathbf{1}, \|f\|=1} f^T L f.$$

因此 Fiedler vector 可以解釋為：

除了 constant function 之外，在 graph 上最 smooth 的 unit vector 。

這個概念必須成為專題的重要數學解釋。

18. 為什麼 Fiedler Vector 可以 Partition ?

假設 graph 存在：

```

Dense Cluster A
  |
weak bridge
  |
Dense Cluster B

```

因為 cluster 內部 edges 很多，所以要讓：

$$f^T L f$$

保持小，cluster 內的 vertices 會傾向具有相近的 $f(i)$ 。

例如：

$$f(i) \approx -c$$

對 cluster A，

而：

$$f(i) \approx +c$$

對 cluster B。

最大的 variation 可以集中在 graph 中 edge 最少的 bottleneck。

因此 Fiedler vector 自然會揭露：

weak cut / bottleneck.

這就是 spectral partitioning 的基本直覺。

19. Predefined Toy Graphs

除了讓使用者自行畫 graph，也提供 predefined examples。

至少包含：

Path Graph

展示 low/high frequency eigenvectors。

Cycle Graph

展示 symmetry 與 spectrum。

Complete Graph

展示高度 connectivity。

Star Graph

展示中心 node 的結構。

Barbell Graph

非常適合展示 Fiedler vector。

Two Dense Clusters + Bridge

展示 algebraic connectivity 與 bottleneck。

Disconnected Graph

展示 zero eigenvalue multiplicity。

Grid Graph

展示較有 geometry 感的 eigenvectors。

20. Bridge Strength Experiment

預設 graph：

Dense Cluster A

|

Dense Cluster B

提供 slider：

Bridge Weight

0 ————— 1

修改中間 edge weight：

$$w.$$

即時觀察：

$$w \rightarrow \lambda_2 \rightarrow v_2.$$

讓使用者理解：

$$\text{weaker connection} \Rightarrow \text{smaller } \lambda_2.$$

21. Module C — Geometric Graph Lab

這個模式與 Free Graph Mode 不同。

此時 node position 代表真正的幾何座標：

$$x_i \in \mathbb{R}^2.$$

22. Point Creation

使用者可以：

- 在 2D plane 點擊新增 point
- 刪除 point
- 拖動 point
- 使用 predefined dataset

例如：

- Two Moons
 - Concentric Circles
 - Gaussian Blobs
 - Swiss-roll projection
 - manually created points
-

23. Geometry → Graph

提供不同方式建立 graph。

k-Nearest Neighbor Graph

使用者調整：

k .

ϵ -Neighborhood Graph

若：

$$\|x_i - x_j\| < \epsilon$$

則建立 edge。

使用者可以調整：

ϵ .

Gaussian Similarity Graph

例如：

$$w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right).$$

使用者可以調整：

$$\sigma.$$

24. Geometric Graph Visualization

畫面同時顯示：

Original Points

以及

Constructed Graph

讓使用者看到：

Euclidean geometry

如何被轉換成：

graph connectivity.

25. Spectral Embedding

對 Laplacian 計算 eigenvectors。

對 k clusters：

$$U = [v_1, \dots, v_k]$$

或根據 Laplacian type 使用適當 eigenvectors。

每個 node i 被映射為：

$$z_i = (v_1(i), \dots, v_k(i)).$$

稱為：

spectral embedding.

對 $k = 2$ 或 3 時，可以直接視覺化 embedding。

26. K-Means vs Spectral Clustering

提供同一組資料的比較。

左側：

K-Means in Original Space

右側：

Spectral Clustering

例如 Two Moons。

展示：

Original Space $\rightarrow$ K-Means

與：

Original Space $\rightarrow$ Similarity Graph $\rightarrow L \rightarrow$ Spectral Embedding $\rightarrow$ K-Means.

目的不是只比較 accuracy。

而是解釋：

Spectral clustering 改變了資料的 representation。

27. Clustering Controls

使用者可以調整：

- number of clusters k
- graph construction method
- number of nearest neighbors
- ε
- Gaussian kernel σ
- Laplacian type
- random seed

並即時更新結果。

28. Module D — Eigenfunction / Mesh Lab

此模組負責建立：

Graph Laplacian

與

Continuous Laplacian

之間的概念連結。

不需要深入證明 PDE 或 Functional Analysis。

重點放在視覺直覺。

29. Continuous Analogy

說明 continuous Laplacian eigenvalue problem：

$$-\Delta\phi = \lambda\phi.$$

例如一維區間上：

$$\phi_k(x) = \sin(kx)$$

具有：

$$\lambda_k = k^2.$$

因此：

$$\lambda \text{ small}$$

對應 low-frequency mode，

而：

$$\lambda \text{ large}$$

對應 high-frequency oscillation。

30. Graph Fourier Interpretation

Graph Laplacian：

$$L = U\Lambda U^T$$

其中：

$$U = [v_1, \dots, v_n].$$

Laplacian eigenvectors 可以被理解為：

Graph Fourier basis.

因此：

$$v_1, v_2, \dots$$

可以視為 graph 上由 low frequency 到 high frequency 的 modes。

31. Eigenvector Gallery

對某一張 graph：

例如：

- path graph
- grid graph
- mesh

顯示：

$$v_1, v_2, v_3, \dots, v_k.$$

讓使用者切換 eigenvector index。

每個 vertex 根據：

$$v_k(i)$$

上色。

使用者應該可以直接看到：

Low k
→ smooth variation

High k
→ rapid oscillation

32. Continuous / Discrete Energy Comparison

說明 continuous Dirichlet energy :

$$\int_{\Omega} |\nabla f(x)|^2 dx$$

與 discrete graph energy :

$$f^T L f = \frac{1}{2} \sum_{i,j} w_{ij} (f(i) - f(j))^2.$$

兩者都衡量 function 的 variation 。

專題只需解釋 :

$$\text{continuous smoothness} \leftrightarrow \text{graph smoothness}$$

不需要深入 measure theory 或 functional analysis 。

33. 3D Mesh Experiment

至少提供一個簡單公開 mesh 。

例如 :

- Bunny
- Armadillo
- Sphere
- Torus
- simple synthetic mesh

對 mesh 建立適合的 discrete Laplacian 。

MVP 可以先使用簡單 graph Laplacian 。

進階版本可以加入 :

- cotangent Laplacian
- mass matrix
- generalized eigenvalue problem

例如 :

$$L\phi = \lambda M\phi.$$

34. Mesh Eigenfunction Visualization

將 eigenvector :

$$v_k(i)$$

映射到 mesh vertex color 。

使用者可以調整 :

Eigenfunction Index

1 ————— 30

觀察 :

$$\lambda_k$$

增加時，eigenfunction 如何逐漸產生更高頻率的 oscillation 。

35. Optional Graph Fourier Lab

若時間足夠，可以加入簡單 graph signal 。

例如給每個 node :

$$x_i.$$

計算 graph Fourier transform :

$$\hat{x} = U^T x.$$

再展示 :

$$x = U \hat{x}.$$

使用者可以看到 :

哪些 graph frequencies 組成原始 signal 。

36. Optional Spectral Filtering

可以進一步加入 :

$$g(L)x = Ug(\Lambda)U^T x.$$

例如 low-pass filter :

降低 high-frequency components。

觀察 filtered graph signal 變得 smoother。

這可以作為：

Spectral Graph Theory → Graph Neural Network

的最後橋樑。

37. Mathematical Explanation Layer

每個模組都需要包含一個：

Why does this work?

或：

Math Behind the Visualization

區塊。

避免網站只剩下視覺效果。

38. 每個數學區塊的建議結構

Definition

先給公式。

Intuition

用簡單語言解釋。

Visualization

讓使用者實際操作。

Observation

指出應該觀察什麼。

Connection

說明與後續 spectral methods / ML 的關係。

39. Suggested Main Navigation

首頁：

Spectral Graph Lab

How does the spectrum of a graph encode
its geometry and connectivity?

下面四個入口：

01 Graph Playground
02 Fiedler & Connectivity
03 Spectral Clustering
04 Eigenfunctions & Graph Fourier

40. Suggested Graph Playground Layout

左側：

Graph Controls

Add Node
Add Edge
Delete
Edge Weight
Laplacian Type
Load Example
Reset

中央：

Interactive Graph

右側：

Spectral Information

λ_1

λ_2

λ_3

...

Algebraic Connectivity

Connected Components

下方：

Adjacency Matrix

Degree Matrix

Graph Laplacian

以及：

Fiedler Vector Visualization

41. Suggested Spectral Clustering Layout

左側：

Dataset

Graph Construction

k

ϵ

σ

Clusters

Laplacian Type

中央：

Original Data

右側：

Similarity Graph

下一排：

Spectral Embedding

以及：

K-Means vs Spectral Clustering

42. Suggested Mesh Layout

左側：

Mesh
Laplacian Type
Eigenvector Index

中央：

Interactive 3D Mesh

右側：

λ_k

Frequency Level
Smoothness Energy

43. MVP 開發順序

不要一開始全部完成。

Phase 1 — Graph Playground

先完成：

- predefined graph
- graph visualization
- adjacency matrix

- degree matrix
- Laplacian
- eigenvalues
- λ_2
- Fiedler vector
- Fiedler coloring
- Fiedler partition

這應該先成為一個完全可用的版本。

Phase 2 — Interactive Graph Editing

加入：

- add node
 - delete node
 - add edge
 - delete edge
 - modify edge weight
 - drag node
-

Phase 3 — Connectivity Experiments

加入：

- connected component experiment
 - bridge weight slider
 - barbell graph
 - Fiedler threshold
 - smoothness energy demo
-

Phase 4 — Spectral Clustering

加入：

- Two Moons
 - Circles
 - kNN graph
 - epsilon graph
 - Gaussian similarity graph
 - spectral embedding
 - K-Means comparison
-

Phase 5 — Eigenvector Gallery

加入：

- path graph
 - grid graph
 - first k eigenvectors
 - low-frequency / high-frequency visualization
-

Phase 6 — Mesh Lab

加入：

- 3D mesh
 - mesh Laplacian
 - eigenfunction coloring
 - eigenfunction slider
-

Phase 7 — Graph Fourier Extension

若時間允許，再加入：

- graph signal
- graph Fourier transform
- frequency spectrum
- low-pass filtering

這部分不是 MVP 必需。

44. 技術架構建議

核心數學計算應該與 UI 分離。

例如概念上分成：

```
graph/  
  laplacian  
  spectrum  
  fiedler  
  clustering  
  graph_signal  
  
visualization/  
  graph_plot
```

```
eigenvalue_plot
eigenvector_plot
embedding_plot
mesh_plot

datasets/
  toy_graphs
  point_clouds
  meshes

ui/
  playground
  connectivity
  clustering
  mesh
```

不要求一定使用這個 file structure。

重點是：

數學函數不要和 Streamlit UI logic 寫死在一起。

45. Core Mathematical Functions

核心邏輯至少應包含：

Graph Construction

輸入：

$$V, E, W$$

輸出 adjacency representation。

Laplacian

支援：

- unnormalized
- normalized symmetric
- random walk

Spectrum

輸出：

$$\lambda_i, v_i.$$

Algebraic Connectivity

輸出：

$$\lambda_2.$$

需要處理 disconnected graph。

Fiedler Vector

輸出：

$$v_2.$$

Graph Energy

計算：

$$f^T L f.$$

Spectral Embedding

輸出前 k 個 relevant eigenvectors。

Spectral Clustering

embedding 後執行 K-Means。

46. 數值計算注意事項

eigenvalues 實際運算可能出現：

$$-10^{-15}$$

之類的 floating-point error。

因此需要 tolerance。

例如理論上：

$$\lambda = 0$$

不能因為數值誤差顯示：

-0.00000000000002

應該視為 zero。

47. Eigenvector Sign Ambiguity

Eigenvector :

$$v$$

與 :

$$-v$$

代表同一個 eigenspace direction 。

因此 Fiedler vector visualization 不應將 sign reversal 視為不同結果 。

例如 :

red / blue

整體交換是正常現象 。

README 中需要簡單說明 eigenvector sign ambiguity 。

48. Repeated Eigenvalues

當 eigenvalue multiplicity 大於 1 時 :

對應 eigenvectors 並不唯一 。

因此某些 highly symmetric graphs :

例如 cycle 或 complete graph ,

不同數值 solver 可能產生不同 basis 。

這不是 bug 。

README 或 UI 可以簡單說明 :

Eigenvectors inside a repeated eigenspace are not unique.

49. Disconnected Graph 處理

如果 graph disconnected :

$$\lambda_2 = 0$$

Fiedler vector interpretation 需要特別處理。

UI 應該提示：

The graph is disconnected.

The multiplicity of eigenvalue 0
corresponds to the number of connected components.

而不是硬套一般 connected graph 的 Fiedler partition。

50. Weighted Graph 處理

如果 edge 有 weight：

$$w_{ij},$$

則：

$$A_{ij} = w_{ij}.$$

Degree：

$$D_{ii} = \sum_j w_{ij}.$$

Smoothness：

$$f^T L f = \frac{1}{2} \sum_{i,j} w_{ij} (f_i - f_j)^2.$$

因此 strong edge 會更強烈懲罰：

$$f_i / f_j =$$

這個概念可以透過 bridge weight experiment 展示。

51. README 故事線

README 不應該只寫如何執行程式。

建議按照以下故事：

1. Why Spectral Graphs?

Graph 看似只是：

$$V, E.$$

但其 Laplacian spectrum 可以揭露：

- connectivity
 - bottlenecks
 - clusters
 - graph geometry
-

2. Graph Laplacian

介紹：

$$L = D - A.$$

3. Spectrum and Connectivity

介紹：

$$\lambda_1, \lambda_2, \dots$$

與：

$$\lambda_2.$$

4. Fiedler Vector

解釋：

$$Lv_2 = \lambda_2 v_2.$$

以及：

$$\lambda_2 = \min_{f \perp 1, \|f\|=1} f^T L f.$$

5. Graph Smoothness

說明：

$$f^T L f = \frac{1}{2} \sum_{ij} w_{ij} (f_i - f_j)^2.$$

6. Spectral Clustering

說明：

Graph $\rightarrow$ Spectral Embedding $\rightarrow$ Clustering.

7. Graph Fourier View

說明：

$$L = U \Lambda U^T$$

與：

$$U$$

作為 graph Fourier basis。

8. From Spectral Graph Theory to Graph Learning

最後連接：

Graph Laplacian $\rightarrow$ Graph Fourier Analysis $\rightarrow$ Spectral Convolution $\rightarrow$ Graph Neural Networks.

52. 專題完成判準

最低完成標準：

Graph Construction

使用者可以自己建立或修改 graph。

Laplacian

graph 改變後即時計算：

$$A, D, L.$$

Spectrum

正確顯示 eigenvalues。

Algebraic Connectivity

正確計算：

$$\lambda_2.$$

Fiedler Vector

可以將：

$$v_2$$

映射到 graph nodes。

Partition

能夠透過 Fiedler vector 進行二分。

Connectivity Experiment

使用者修改 bridge 時能看到：

$$\lambda_2$$

改變。

Spectral Clustering

至少有一個資料集展示：

K-Means

與：

Spectral Clustering

的差異。

Mathematical Explanation

每個主要結果都有相對應的數學解釋。

Deployment

有可直接使用的 online demo。

53. 不應該做成什麼

本專題不應該只是：

Upload Dataset
↓
Run Spectral Clustering
↓
Show Result

這樣會變成普通的 clustering demo。

也不應該只是：

Graph
↓
Show Eigenvalues

而缺少 interpretation。

真正的重點是：

Manipulate → Observe → Explain

也就是：

使用者改變 graph，

看到 spectrum 改變，

再理解為什麼 spectrum 會改變。

54. 最終專題故事

整個 Spectral Graph Lab 應該形成以下完整敘事：

Graph → Matrix → Spectrum → Geometry → Clustering → Graph Learning

第一階段：

Graph → L

理解 Graph Laplacian。

第二階段：

$L \rightarrow \lambda_2, v_2$

理解 algebraic connectivity 與 Fiedler vector。

第三階段：

$$v_2 \rightarrow \text{partition}$$

理解 spectral partitioning。

第四階段：

$$v_1, \dots, v_k \rightarrow \text{spectral embedding}$$

理解 spectral clustering。

第五階段：

$$v_k \rightarrow \text{graph frequencies}$$

理解 eigenvectors 作為 graph Fourier modes。

最後：

$$\text{Spectral Graph Theory} \rightarrow \text{Graph Learning.}$$

55. 專題最終定位

Spectral Graph Lab 不應被定位成：

An implementation of spectral clustering.

而應定位成：

An interactive mathematical laboratory for exploring how Graph Laplacian eigenvalues and eigenvectors encode graph connectivity, geometry, smoothness, and clustering structure.

最終希望使用者不只是知道：

Fiedler vector 可以拿來分群。

而是真的理解：

Why does it work?

也就是：

Spectral methods reveal graph structure because Laplacian eigenvectors solve constrained smoothness problem

這就是整個專題最核心的數學思想。